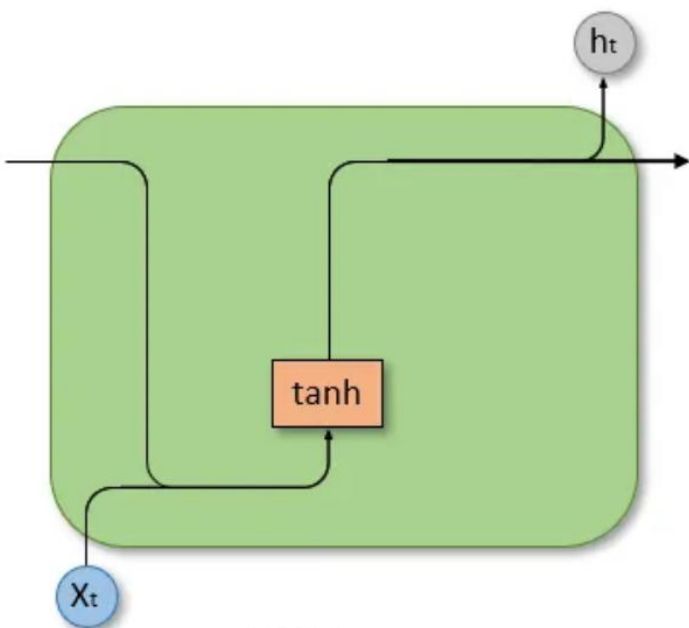
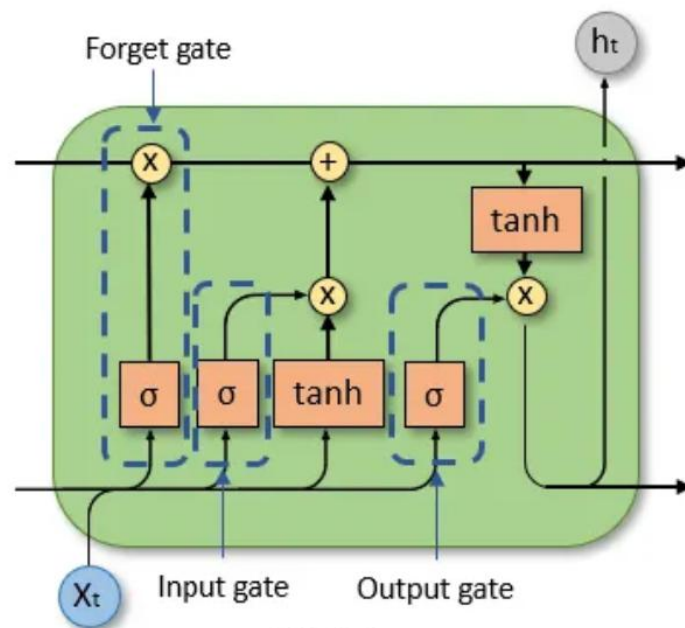




RNN



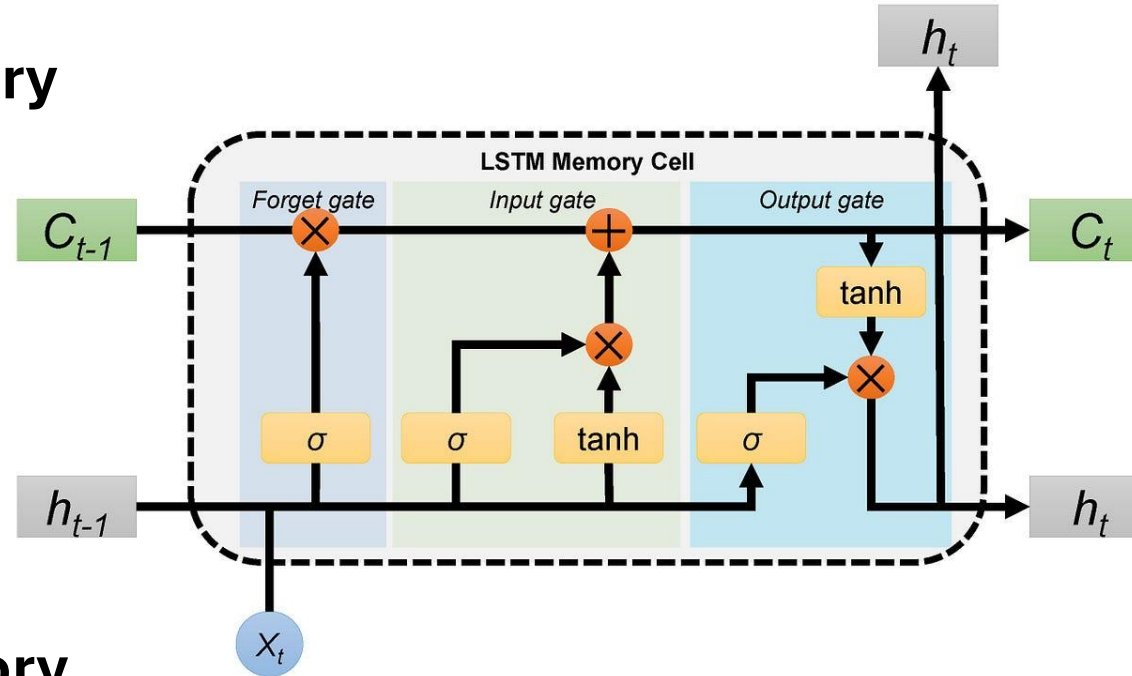
LSTM

LSTM resolve the vanishing gradient by

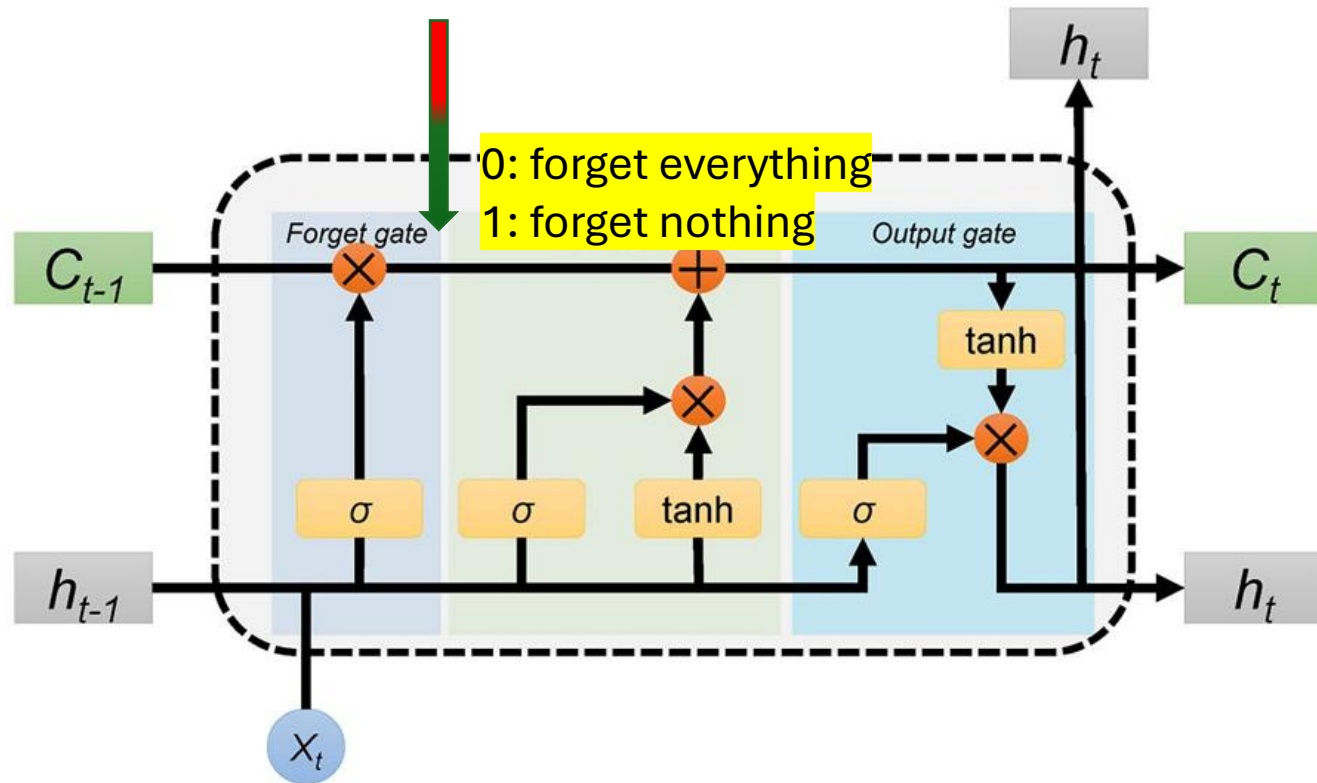
- 1- Forget irrelevant information
- 2-Add/update new information
- 3-Pass updated information

C → Long term memory

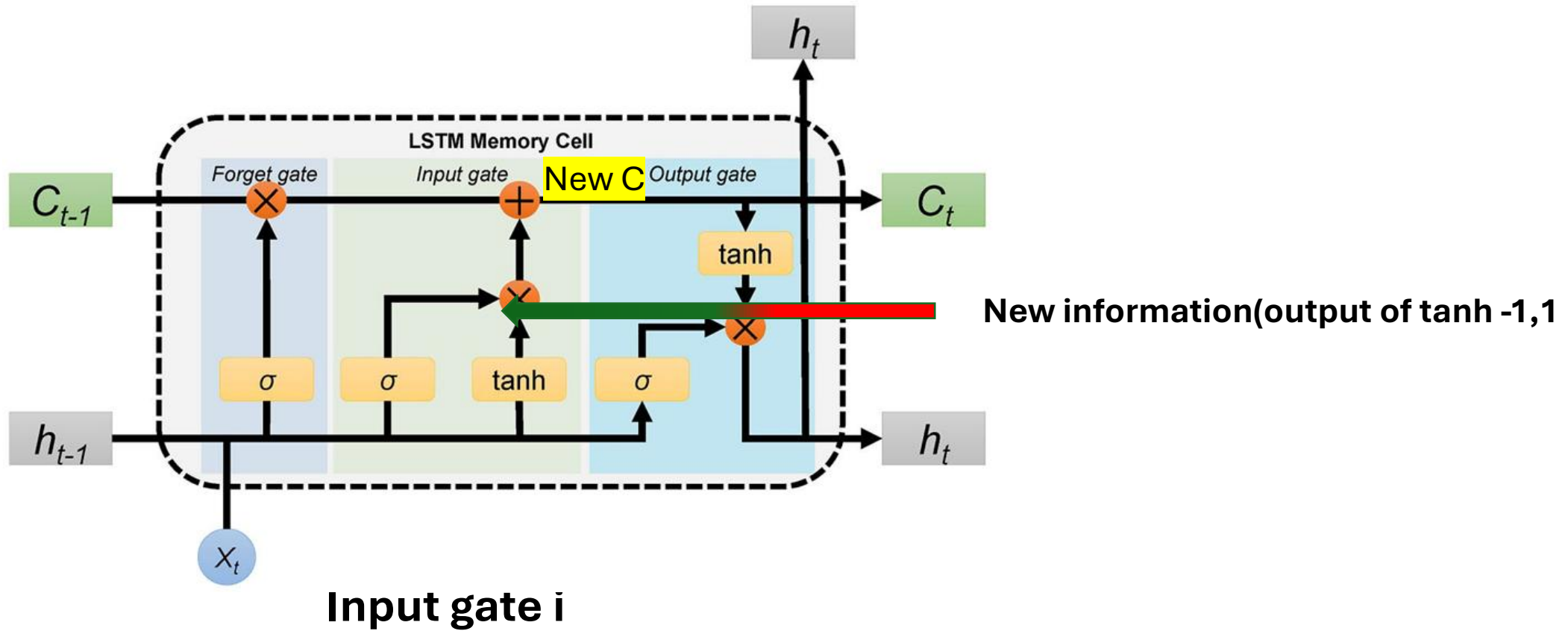
h → Short term memory

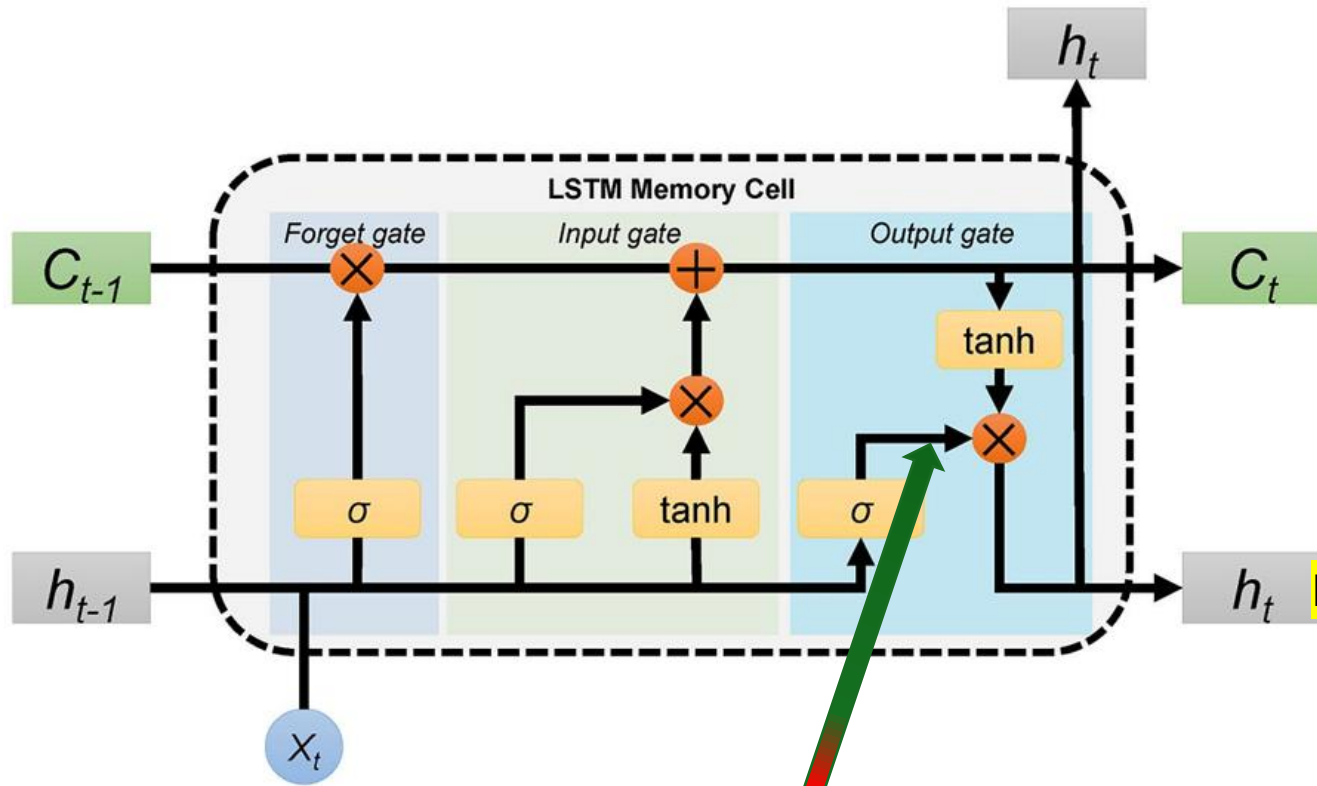


Forget gate $f \rightarrow$ Short term memory



Updating the cell state





hidden state is a function of Long term memory (C_t)

Output gate o

An LSTM cell maintains two key pieces of state:

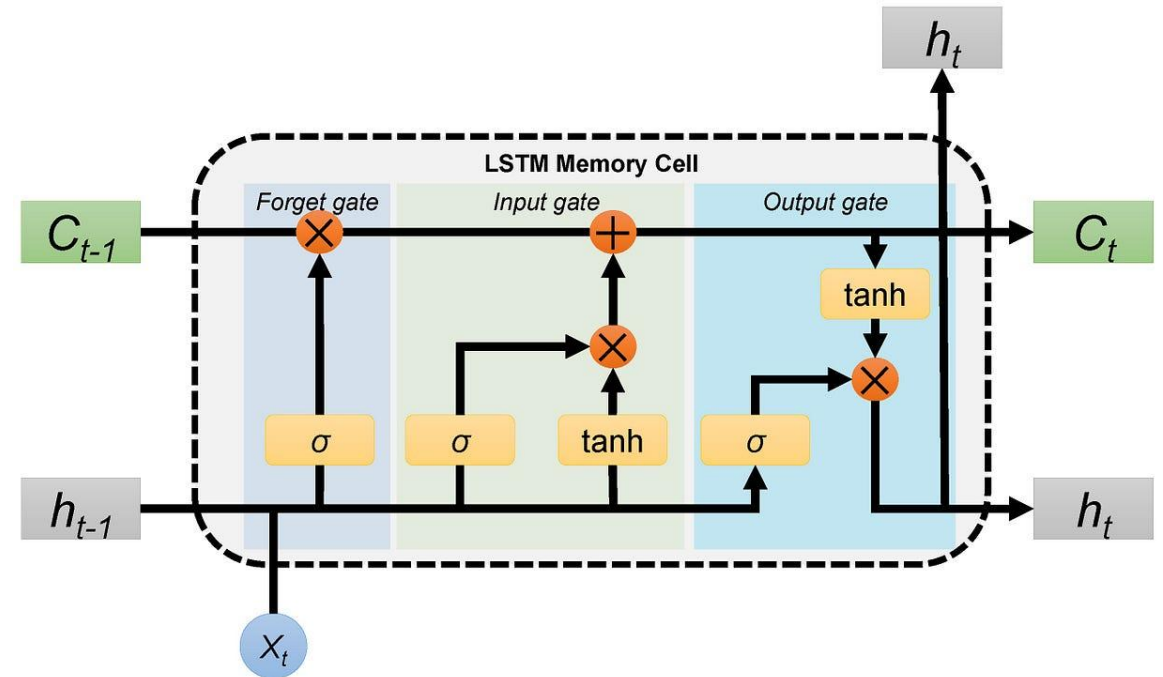
- Hidden state h_t
- Cell state c_t

Each time step computes:

1. Forget gate: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
2. Input gate: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
3. Candidate: $\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$
4. Cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
5. Output gate: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
6. Hidden state: $h_t = o_t \odot \tanh(c_t)$

We will use:

- Input size: 1
- Hidden size: 1
- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Tanh: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

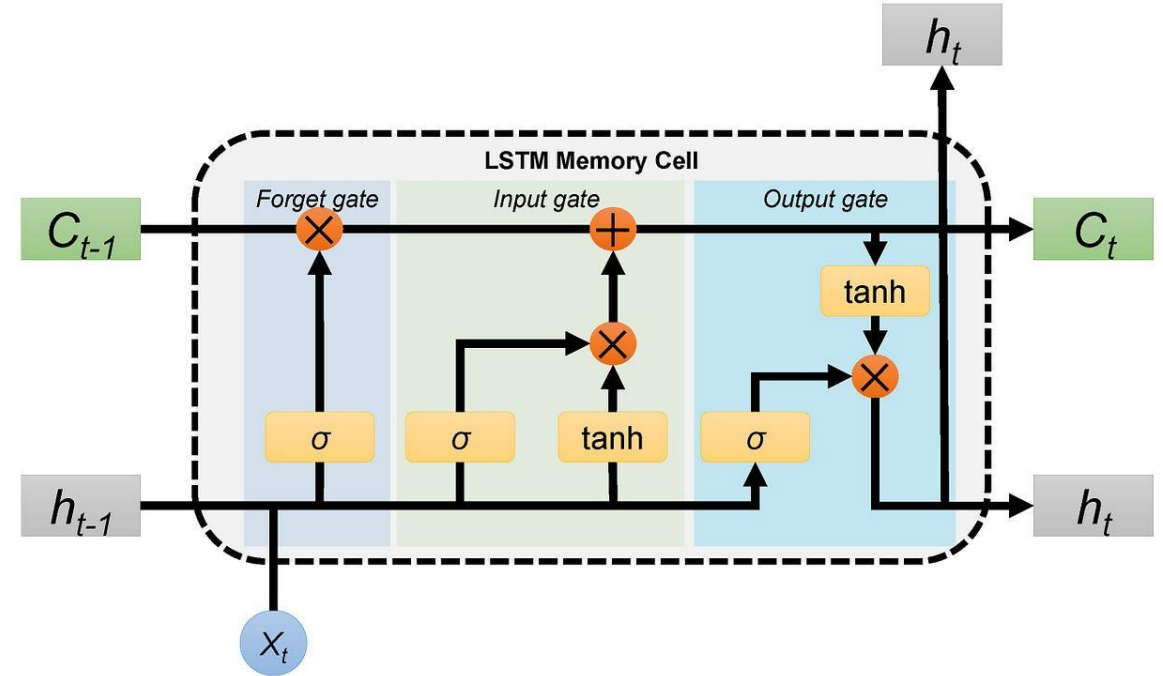


Assume:

- Input sequence: $x = [0.5, 0.1, -0.3]$
- Initial states: $h_0 = 0, c_0 = 0$
- All weights = 1.0
- All biases = 0.0

So for any gate:

$$z = W \cdot h_{t-1} + W \cdot x_t + b = h_{t-1} + x_t$$



Previous state:

$$h_0 = 0, c_0 = 0$$

Compute gates:

- Forget gate:

$$f_1 = \sigma(h_0 + x_1) = \sigma(0 + 0.5) = \sigma(0.5) \approx 0.622$$

- Input gate:

$$i_1 = \sigma(h_0 + x_1) = \sigma(0.5) \approx 0.622$$

- Candidate:

$$\tilde{c}_1 = \tanh(h_0 + x_1) = \tanh(0.5) \approx 0.462$$

- Cell state:

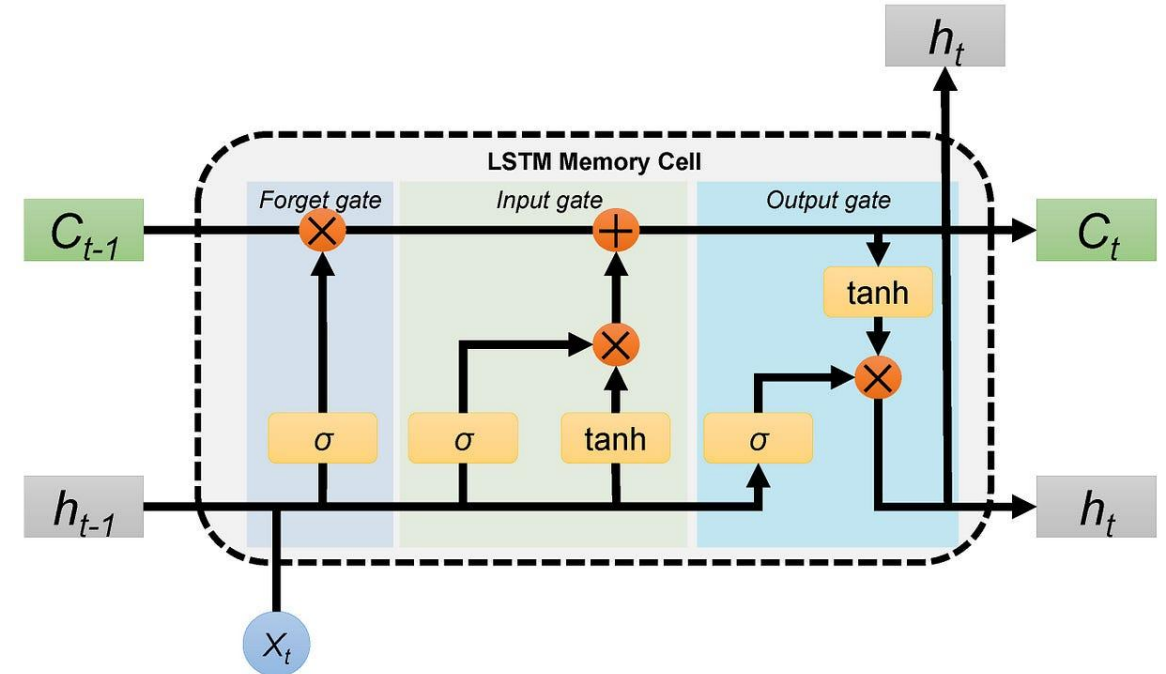
$$c_1 = f_1 \cdot c_0 + i_1 \cdot \tilde{c}_1 = 0 + 0.622 \cdot 0.462 \approx 0.287$$

- Output gate:

$$o_1 = \sigma(h_0 + x_1) = \sigma(0.5) \approx 0.622$$

- Hidden state:

$$h_1 = o_1 \cdot \tanh(c_1) = 0.622 \cdot \tanh(0.287) \approx 0.622 \cdot 0.280 = 0.174$$

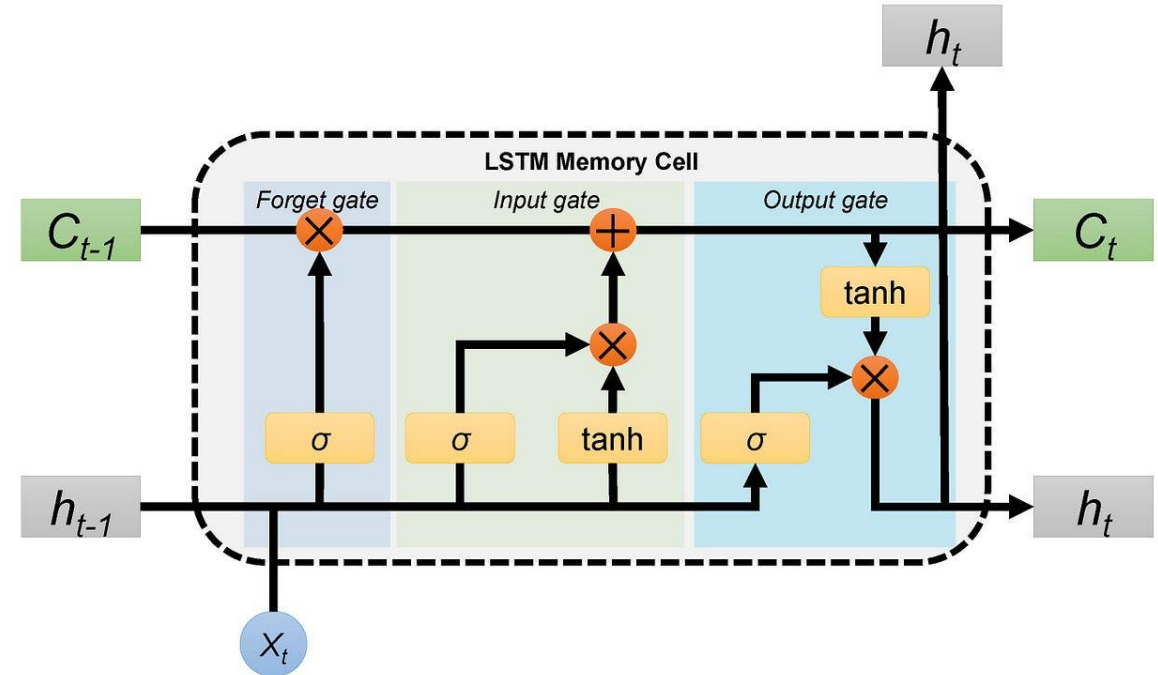


Time Step 1: $t = 1$,
Input: $x_1 = 0.5$

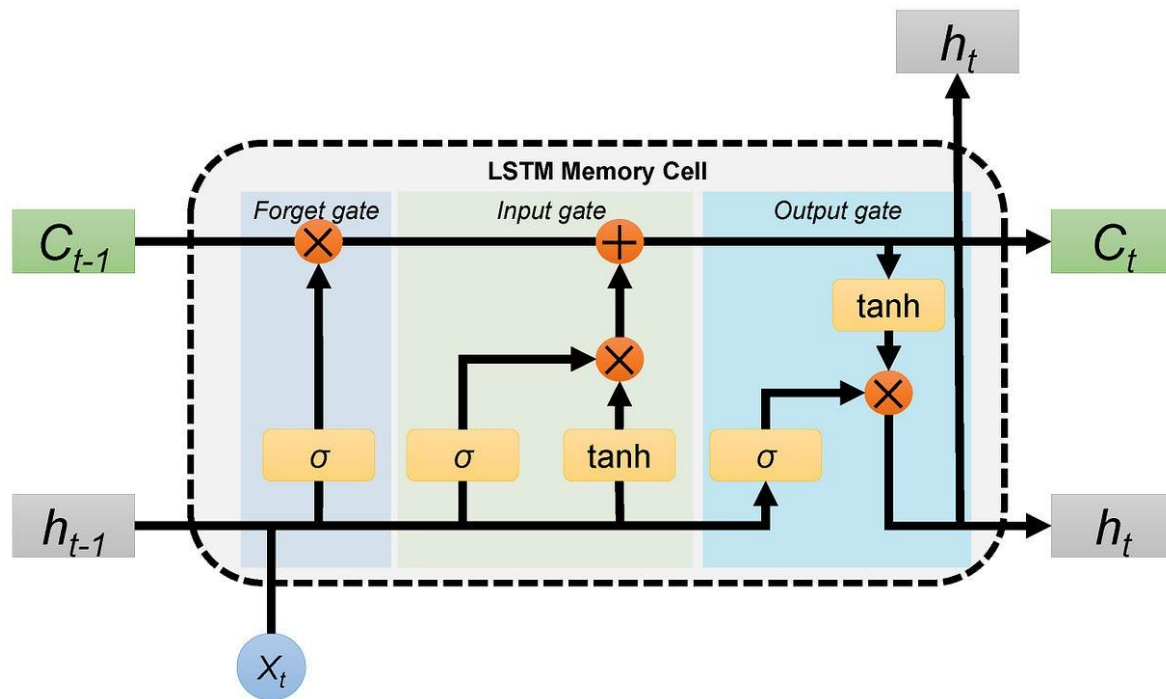
Previous state:

$$h_1 \approx 0.174, c_1 \approx 0.287$$

- Forget gate:
$$f_2 = \sigma(h_1 + x_2) = \sigma(0.174 + 0.1) = \sigma(0.274) \approx 0.568$$
- Input gate:
$$i_2 = \sigma(0.274) \approx 0.568$$
- Candidate:
$$\tilde{c}_2 = \tanh(0.274) \approx 0.267$$
- Cell state:
$$c_2 = f_2 \cdot c_1 + i_2 \cdot \tilde{c}_2 = 0.568 \cdot 0.287 + 0.568 \cdot 0.267 \approx 0.163 + 0.152 = 0.315$$
- Output gate:
$$o_2 = \sigma(0.274) \approx 0.568$$
- Hidden state:
$$h_2 = o_2 \cdot \tanh(c_2) = 0.568 \cdot \tanh(0.315) \approx 0.568 \cdot 0.305 = 0.173$$



- Time Step 2: $t=2$, Input: $x_2=0.1$ $=0.1$



Previous state:

$$h_2 \approx 0.173, c_2 \approx 0.315$$

- Forget gate:

$$f_3 = \sigma(h_2 + x_3) = \sigma(0.173 + (-0.3)) = \sigma(-0.127) \approx 0.468$$
- Input gate:

$$i_3 = \sigma(-0.127) \approx 0.468$$
- Candidate:

$$\tilde{c}_3 = \tanh(-0.127) \approx -0.126$$
- Cell state:

$$c_3 = f_3 \cdot c_2 + i_3 \cdot \tilde{c}_3 = 0.468 \cdot 0.315 + 0.468 \cdot (-0.126) \approx 0.147 - 0.059 = 0.088$$
- Output gate:

$$o_3 = \sigma(-0.127) \approx 0.468$$
- Hidden state:

$$h_3 = o_3 \cdot \tanh(c_3) = 0.468 \cdot \tanh(0.088) \approx 0.468 \cdot 0.088 = 0.041$$

Time Step 3: $t=3$, Input: $x_3=-0.3$ $=-0.3$

Summary table



Time	Input (x_t)	Forget (f_t)	Input (i_t)	Candidate ($\tilde{c}_t$)	Cell (c_t)	Output (o_t)	Hidden (h_t)
1	0.5	0.622	0.622	0.462	0.287	0.622	0.174
2	0.1	0.568	0.568	0.267	0.315	0.568	0.173
3	-0.3	0.468	0.468	-0.126	0.088	0.468	0.041

How to build and train an LSTM model for time series forecasting

Data generation (sine + noise)

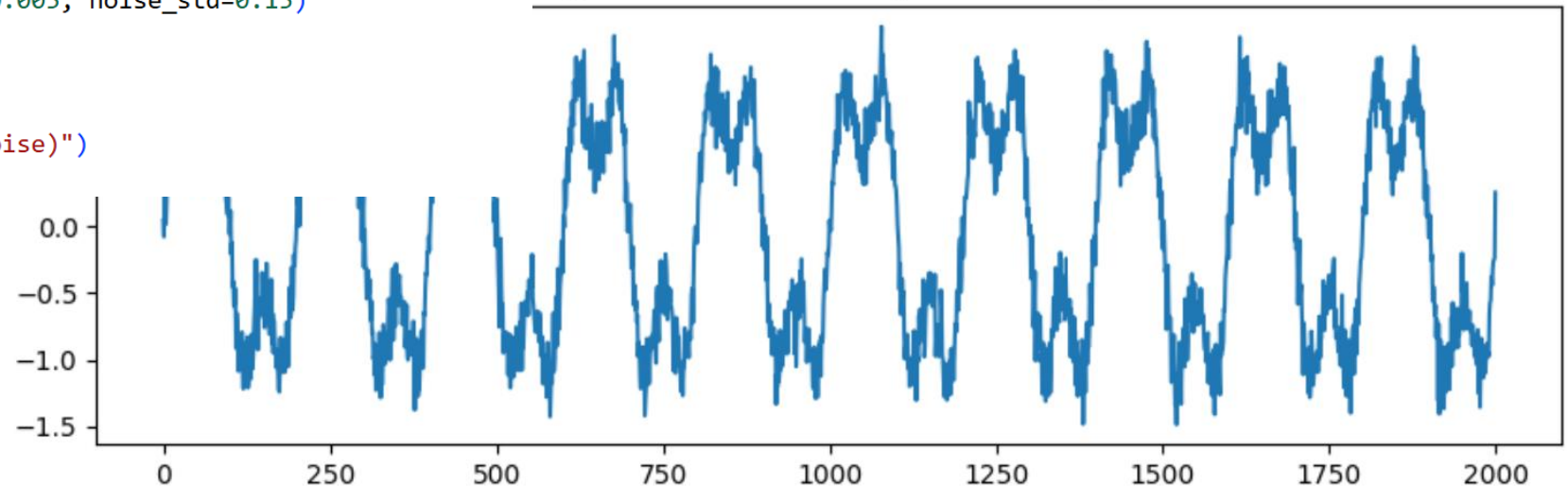
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
import math
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# ----- Create synthetic time series (sine + noise) -----
def make_sine_wave(length=2000, freq=0.01, noise_std=0.2, seed=42):
    rng = np.random.default_rng(seed)
    x = np.arange(length)
    y = np.sin(2 * np.pi * freq * x) + 0.5 * np.sin(2 * np.pi * (freq * 3) * x)
    y += rng.normal(0, noise_std, size=length)
    return y
```

```
series = make_sine_wave(length=2000, freq=0.005, noise_std=0.15)
```

```
plt.figure(figsize=(10,3))
plt.plot(series)
plt.title("Synthetic time series (sine + noise)")
plt.show()
```

Synthetic time series (sine + noise)



Preprocessing and sequence creation

```
# ----- Preprocessing: scale & create sequences -----
def create_sequences(data, seq_len):
    xs, ys = [], []
    for i in range(len(data) - seq_len):
        xs.append(data[i:i+seq_len])
        ys.append(data[i+seq_len])
    X = np.array(xs)
    y = np.array(ys)
    X = X.reshape((X.shape[0], X.shape[1], 1))
    return X, y

train_size = int(len(series) * 0.7)
val_size = int(len(series) * 0.15)

train_series = series[:train_size]
val_series = series[train_size:train_size+val_size]
test_series = series[train_size+val_size:]

scaler = MinMaxScaler(feature_range=(0, 1))
train_series_scaled = scaler.fit_transform(train_series.reshape(-1,1)).flatten()
val_series_scaled = scaler.transform(val_series.reshape(-1,1)).flatten()
test_series_scaled = scaler.transform(test_series.reshape(-1,1)).flatten()

SEQ_LEN = 40
X_train, y_train = create_sequences(train_series_scaled, SEQ_LEN)
X_val, y_val = create_sequences(np.concatenate([train_series_scaled[-SEQ_LEN:], val_series_scaled]), SEQ_LEN)
X_test, y_test = create_sequences(np.concatenate([val_series_scaled[-SEQ_LEN:], test_series_scaled]), SEQ_LEN)

print("Shapes:", X_train.shape, y_train.shape, X_val.shape, y_val.shape, X_test.shape, y_test.shape)
```

➤ Shapes: (1360, 40, 1) (1360,) (300, 40, 1) (300,) (300, 40, 1) (300,)

Building and training an LSTM model

```
# ----- Build the LSTM model -----
tf.random.set_seed(123)
model = Sequential([
    LSTM(64, input_shape=(SEQ_LEN, 1), return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

➔ /usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: D
super().__init__(**kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	16,896
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 19,009 (74.25 KB)
Trainable params: 19,009 (74.25 KB)
Non-trainable params: 0 (0.00 B)

Train the model

```
# ----- Train the model -----  
callbacks = [EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)]  
history = model.fit(  
    X_train, y_train,  
    validation_data=(X_val, y_val),  
    epochs=200,  
    batch_size=32,  
    callbacks=callbacks,  
    verbose=2  
)
```

```
plt.figure(figsize=(8,3))  
plt.plot(history.history['loss'], label='train loss')  
plt.plot(history.history['val_loss'], label='val loss')  
plt.legend()  
plt.title("Training loss")  
plt.show()
```

43/43 - 1s - 19ms/step - loss: 0.0043 - mae: 0.0518 - val_loss: 0.0037 - val_mae: 0.0481

Epoch 26/200

43/43 - 1s - 20ms/step - loss: 0.0042 - mae: 0.0516 - val_loss: 0.0035 - val_mae: 0.0471

Epoch 27/200

43/43 - 1s - 28ms/step - loss: 0.0042 - mae: 0.0514 - val_loss: 0.0040 - val_mae: 0.0497

Epoch 28/200

43/43 - 1s - 19ms/step - loss: 0.0039 - mae: 0.0498 - val_loss: 0.0036 - val_mae: 0.0473

Epoch 29/200

43/43 - 2s - 40ms/step - loss: 0.0041 - mae: 0.0509 - val_loss: 0.0039 - val_mae: 0.0486

Epoch 30/200

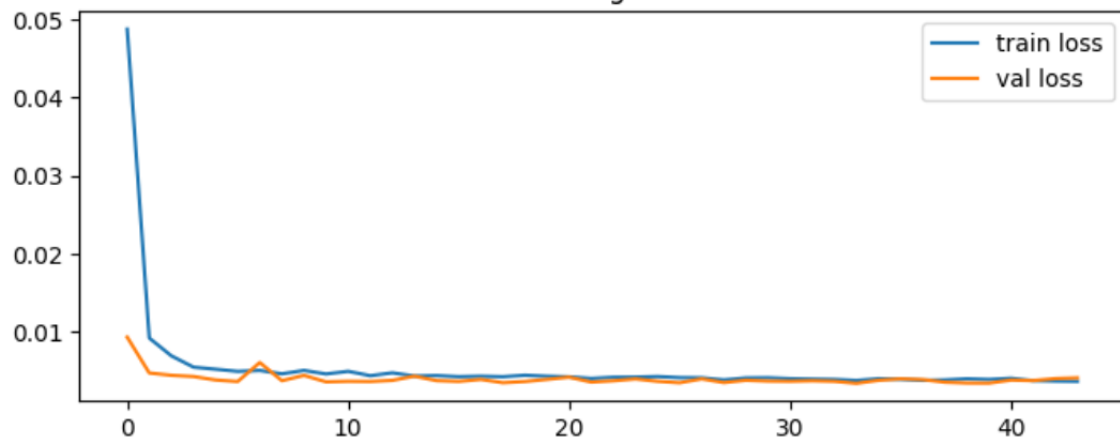
Epoch 31/200

43/43 - 1s - 21ms/step - loss: 0.0037 - mae: 0.0484 - val_loss: 0.0042 - val_mae: 0.0505

342 - mae: 0.0514 - val_loss: 0.0037 - val_mae: 0.0480

341 - mae: 0.0507 - val_loss: 0.0037 - val_mae: 0.0480

Training loss




```
# ----- Evaluate on test set -----
preds_test_scaled = model.predict(X_test).flatten()

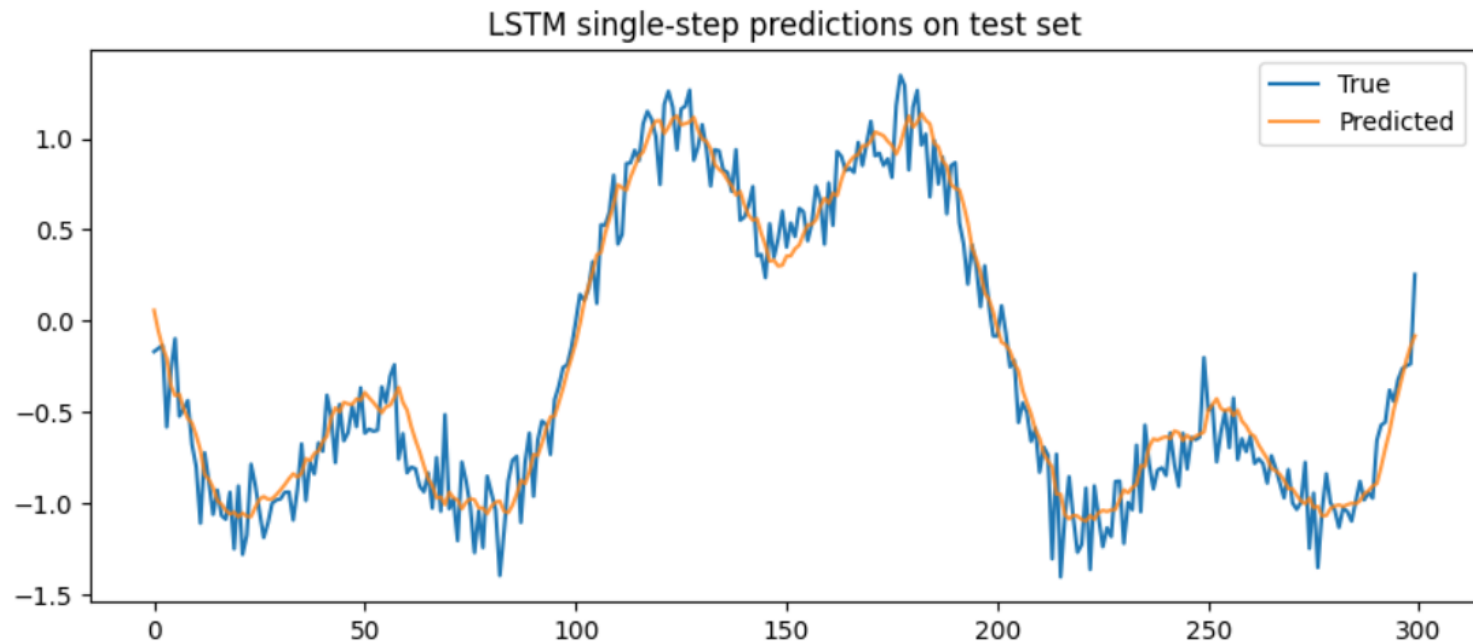
y_test_orig = scaler.inverse_transform(y_test.reshape(-1,1)).flatten()
preds_test_orig = scaler.inverse_transform(preds_test_scaled.reshape(-1,1)).flatten()

mae = mean_absolute_error(y_test_orig, preds_test_orig)
rmse = math.sqrt(mean_squared_error(y_test_orig, preds_test_orig))
print(f"Test MAE: {mae:.4f}, RMSE: {rmse:.4f}")

plt.figure(figsize=(10,4))
plt.plot(y_test_orig, label='True')
plt.plot(preds_test_orig, label='Predicted', alpha=0.8)
plt.legend()
plt.title("LSTM single-step predictions on test set")
plt.show()
```

10/10 ————— 0s 27ms/step

Test MAE: 0.1370, RMSE: 0.1703



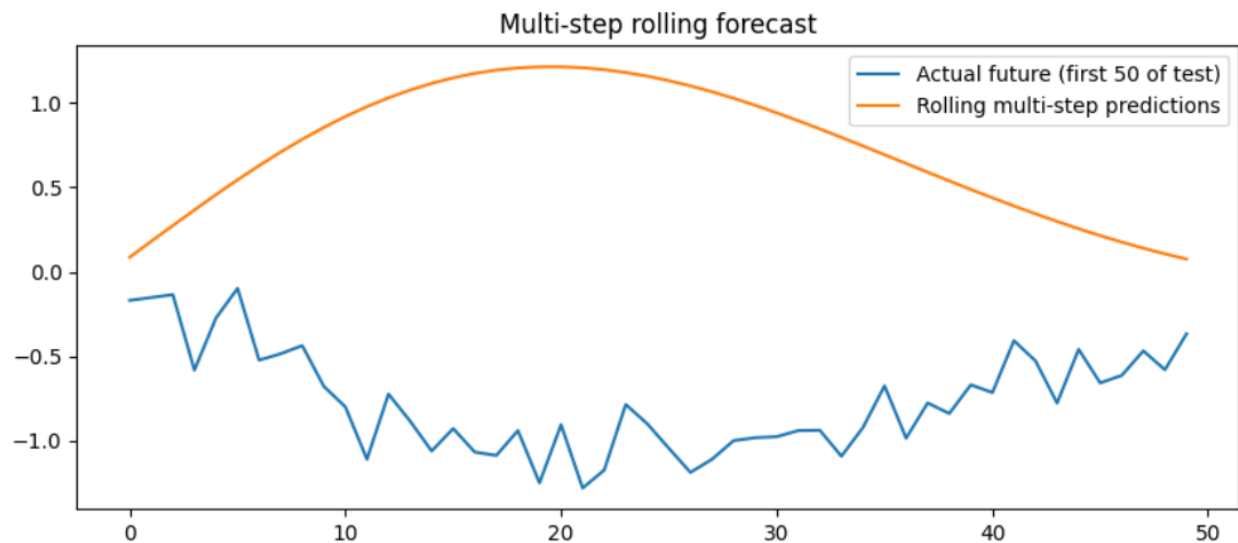
```
# ----- Multi-step rolling forecast -----
```

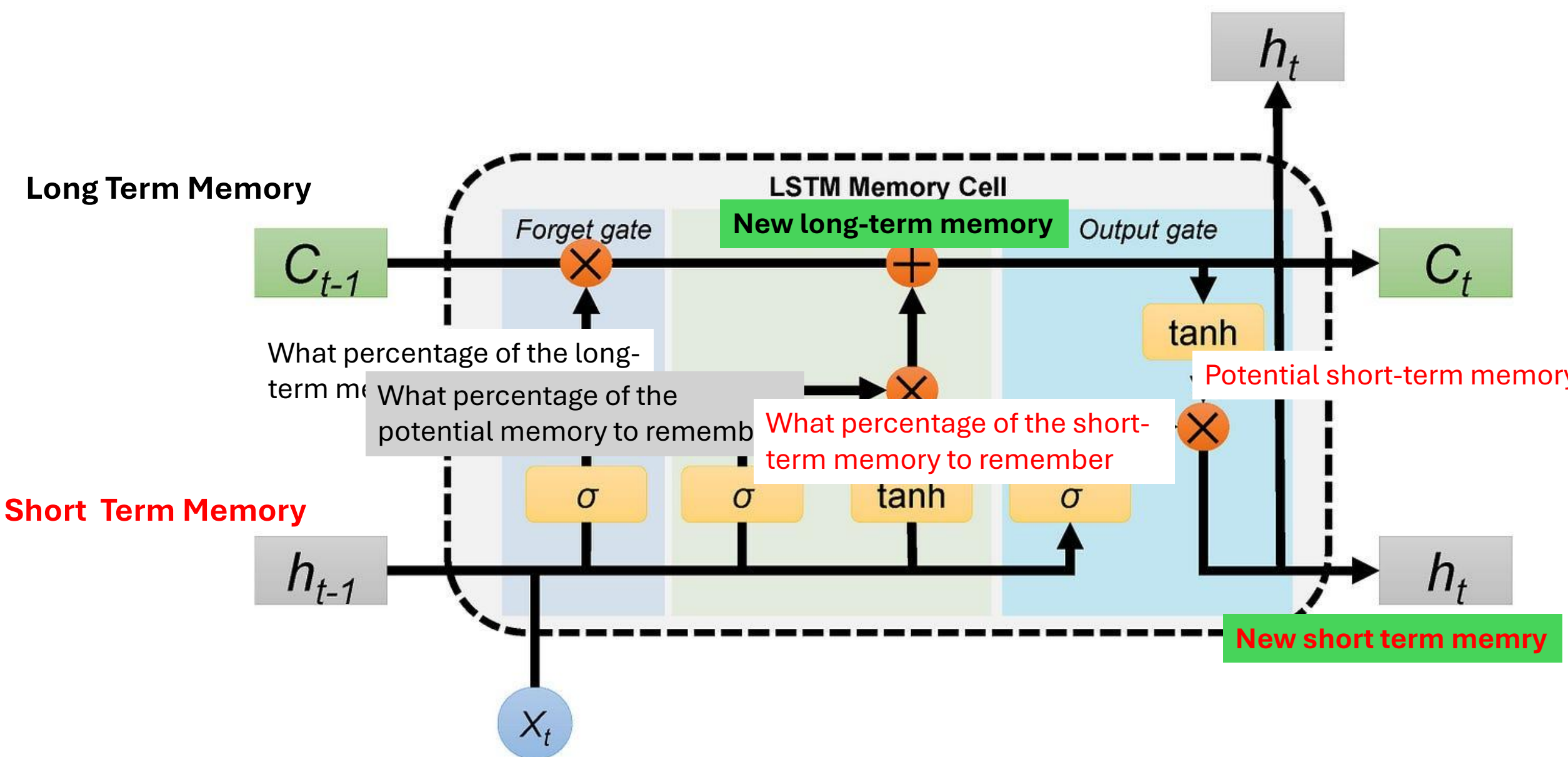
```
def rolling_forecast(last_sequence, model, n_steps, scaler):  
    seq = last_sequence.copy()  
    preds_scaled = []  
    for _ in range(n_steps):  
        x = seq[-SEQ_LEN:].reshape((1, SEQ_LEN, 1))  
        p = model.predict(x)[0,0]  
        preds_scaled.append(p)  
        seq = np.append(seq, p)  
    preds_scaled = np.array(preds_scaled).reshape(-1,1)  
    preds_orig = scaler.inverse_transform(preds_scaled).flatten()  
    return preds_orig
```

```
last_seq = np.concatenate([val_series_scaled[-SEQ_LEN:], test_series_scaled])[-SEQ_LEN:]  
multi_preds = rolling_forecast(last_seq, model, n_steps=50, scaler=scaler)
```

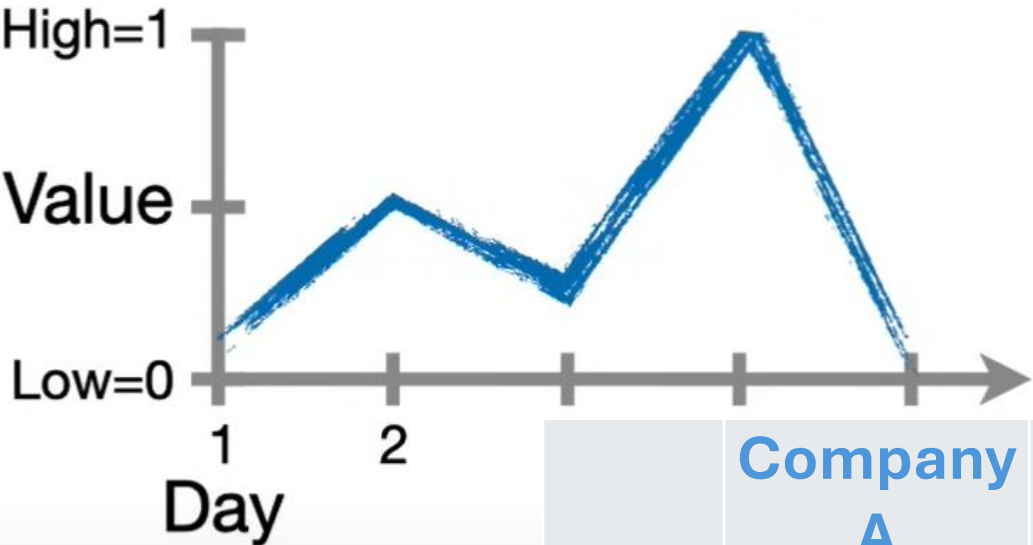
```
plt.figure(figsize=(10,4))  
actual_future = test_series[:50]  
plt.plot(actual_future, label='Actual future (first 50 of test)')  
plt.plot(multi_preds, label='Rolling multi-step predictions')  
plt.legend()  
plt.title("Multi-step rolling forecast")  
plt.show()
```

```
1/1 ----- 0s 42ms/step  
1/1 ----- 0s 39ms/step  
1/1 ----- 0s 42ms/step  
1/1 ----- 0s 38ms/step  
1/1 ----- 0s 43ms/step  
1/1 ----- 0s 41ms/step  
1/1 ----- 0s 43ms/step  
1/1 ----- 0s 40ms/step  
1/1 ----- 0s 45ms/step  
1/1 ----- 0s 40ms/step  
1/1 ----- 0s 39ms/step  
1/1 ----- 0s 41ms/step  
1/1 ----- 0s 42ms/step  
...
```

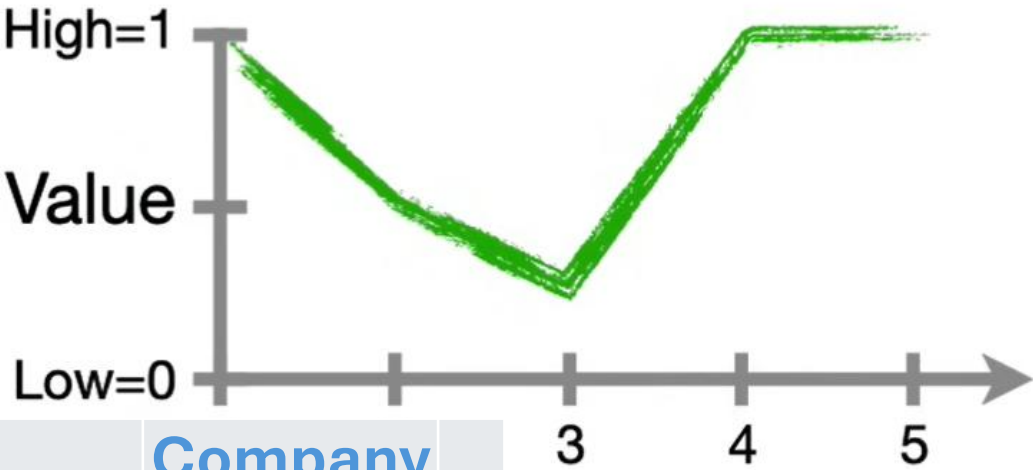




Predict the values of day 5 for both companies A and B based on the values of days 1,2,3,4

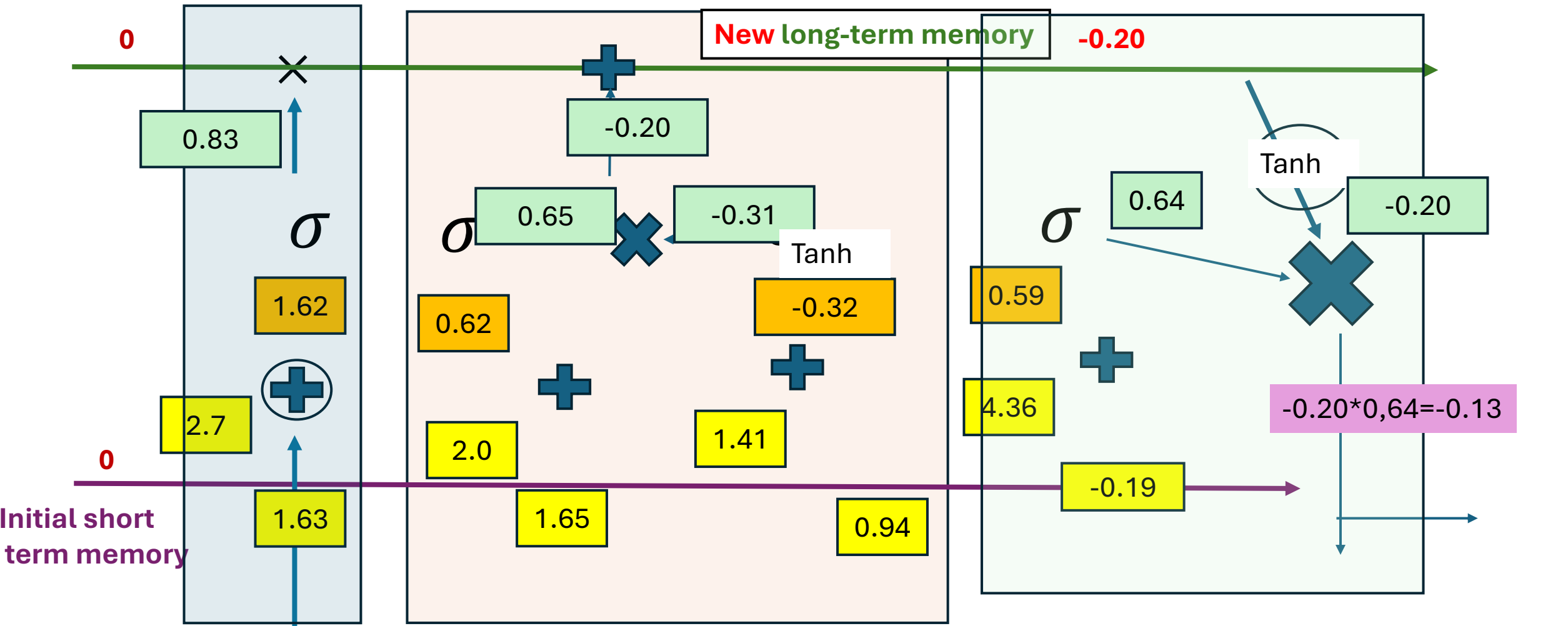


	Company A			
Day	value			
1	0			
2	0.5			
3	0.25			
4	1			
5???	0???			



	Company B	
Day	value	
1	1	
2	0.5	
3	0	
4	1	
5???	1??	

Initial long-term memory



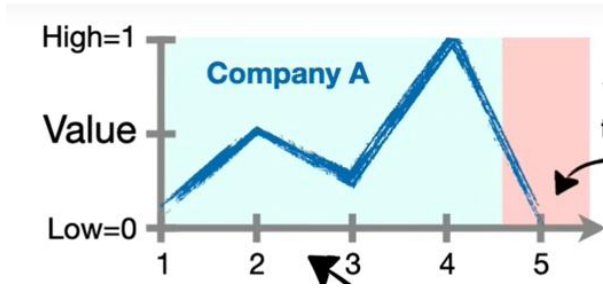
$$(0 \times 2.7 + 0 \times 1.63 + 1.62 = 1.62)$$
$$\text{Sig}(1.62) = 0.83$$

$$(0 \times 1.65 + 0 \times 2 + 0.62 = -0.62)$$
$$\text{sig}(0.62) = 0.65$$

$$(0 \times 0.94 + 0 \times 1.63 + -0.32 = -0.32)$$
$$\text{Tanh}(-0.32) = -0.3106$$

$$(0 \times -0.19 + 0 \times 4.36 + 0.59 = 0.59)$$
$$\text{sig}(0.59) = 0.64$$

$$(\text{Tanh}(-0.20) = -0.20)$$



Initial long-term memory

Updated long term memory

Initial short
term memory

Updated short term memory

Day 1 = 0

Day 2 = 0.5

